How to Present EduTune AI

A step-by-step guide for presenting the project to faculty: slides, what to say, the live demo, and how to answer questions

How to use this guide

- Read Sections 1–2 once to understand the **story** you are telling.
- Build your slides from **Section 3** (one card = one slide) and rehearse the “What to say” lines aloud.
- Practise the **live demo** in Section 4 at least twice on your own laptop.
- Section 5 tells you how to describe the dataset **honestly** — read it carefully.
- Before the day, go through the **checklist** in Section 8.

The project in one line	
What	An AI teaching assistant for AI / ML / DL / GenAI, made by fine-tuning an open-source LLM
How	Instruction tuning with QLoRA: 4-bit frozen base model + LoRA adapters (1.18% of parameters trained)
Proof	Perplexity 8.612 → 2.482 (−71%), side-by-side answers, a working app

Contents

1. The Story You Are Telling 3

1.1 What faculty will be looking for . . . 3

1.2 Time plan (15 minutes) . . . 3

2. Slide Design Tips 4

3. Slide-by-Slide Script 5

4. The Live Demo (4 minutes) 8

5. Talking About the Dataset and AI Assistance Honestly 10

5.1 The facts . . . 10

5.2 How to say it . . . 10

5.3 Things NOT to claim . . . 10

6. Questions Faculty Are Likely to Ask 11

7. Presentation Do's and Don'ts 13

8. Checklist 13

8.1 The day before . . . 13

8.2 Thirty minutes before . . . 13

8.3 Numbers to remember . . . 14

1. The Story You Are Telling

Faculty remember a clear story better than a list of technologies. Every slide should support this single storyline:

Part	Message
Problem	General chatbots don't teach in a consistent, structured way, and fine-tuning a whole LLM normally needs expensive GPUs.
Idea	Take a small open-source LLM and **teach it to be a tutor** by training it on examples of good teaching answers (instruction tuning).
Trick	Use **QLoRA**: shrink the frozen model to 4-bit and train only small LoRA adapters, so it fits on a laptop GPU (17 minutes, 2.81 GB).
Proof	Perplexity dropped by 71%, the model follows teaching formats (e.g. MCQs with answer keys), and the base vs fine-tuned difference is visible live.
Product	A working app: tutor chat with 9 teaching modes, quiz, viva examiner, interview coach, MCQ generator.
Honesty	What it does not do: it doesn't make the model know more facts, so it can still make mistakes.

Your opening sentence (memorise this)

"I took a pretrained language model and specialised it into an AI Teaching Assistant using instruction tuning. I used PEFT with LoRA, and QLoRA to reduce GPU memory requirements. The model was trained on educational instruction-response examples and can now perform different teaching tasks such as simple explanations, analogies, viva preparation, interview preparation, MCQs, and quizzes."

1.1 What faculty will be looking for

- **Is it real fine-tuning?** Yes — show the training script, the trainable-parameter printout and the adapter file.
- **Do you understand it?** Be able to explain LoRA ($h = Wx + (\alpha/r)\cdot BAx$) and why 4-bit saves memory.
- **Did it work?** Show numbers (perplexity) *and* a side-by-side comparison.
- **Are you honest about limitations?** Mention them before they ask — this earns marks.
- **What was your contribution?** Be clear about what you built and how AI tools helped (Section 5).

1.2 Time plan (15 minutes)

Part	Slides	Time
Introduction and problem	1–3	2:00
How it works (architecture, dataset, concepts)	4–7	4:30
Training and results	8–9	2:30
Live demo	10	4:00
Limitations and conclusion	11–12	1:15
Buffer		0:45

If you only have **10 minutes**: merge slides 2–3, skip slide 6 (explain the concepts while showing slide 7) and shorten the demo to steps 1, 2 and 4.

2. Slide Design Tips

- One idea per slide, at most 5 bullets, and large fonts (at least 24 pt for bullets).
- Use the diagrams and charts from `docs/EduTune_AI_Project_Report.pdf` (Figures 1–6) and `results/training_loss.png`.
- Put the **numbers** on the slides; put the **explanations** in what you say.
- Show a real dataset record and real outputs — real material is more convincing than descriptions.
- Keep code off the slides except the 5-line LoRA config (slide 7). Faculty can ask to see the code.
- End with a slide showing the one-sentence summary; leave it on screen during questions.

Files to have open in tabs before you start

- The app in the browser (`streamlit run app.py`), already loaded.
- `data/instruction_dataset.jsonl` in VS Code (to show real records).
- `config.py` (LoRA settings) and `training/train_qlora.py`.
- `results/training_loss.png` and `results/training_summary.json`.
- `docs/EduTune_AI_Project_Report.pdf` as a backup of every figure and number.

3. Slide-by-Slide Script

Each card is one slide: what to put on it (left) and what to say (right). Say it in your own words — don't read it out.

Slide 1 · Title (0:30)

Put on the slide
- EduTune AI — Instruction-Tuned AI Teaching Assistant
- Your name, guide's name, department, date
- Subtitle: Supervised fine-tuning with LoRA and QLoRA

What to say
"Good morning. My project is EduTune AI — an AI teaching assistant that I created by fine-tuning an open-source language model to teach AI and machine learning concepts."

Slide 2 · Problem statement (1:00)

Put on the slide
- General LLMs are not optimised for a teaching style
- Answers vary in structure, depth and format
- Apps can't rely on their output format (e.g. MCQ answer keys)
- Full fine-tuning of a 1.5B model needs ~25 GB of GPU memory

What to say
"Chatbots like ChatGPT are general-purpose. When a student asks one to explain LSTM to a beginner, or to act as a viva examiner, the style changes from question to question. Specialising a model means fine-tuning it, but full fine-tuning needs data-centre GPUs, which students don't have."

Slide 3 · Objectives (0:30)

Put on the slide
- Build an instruction dataset for AI/ML/DL/GenAI
- Fine-tune with PEFT + LoRA, using QLoRA (4-bit)
- Run everything on a laptop GPU
- Compare base vs fine-tuned honestly
- Build a teaching application

What to say
"My objectives were to create a teaching dataset, actually fine-tune a model — not just write prompts — on my own laptop, measure whether it improved, and wrap it in a usable application."

Slide 4 · Architecture (1:00)

Put on the slide
- Knowledge base → generator → 840 examples
- Cleaning + 90/10 split
- 4-bit base model + LoRA → QLoRA training
- Adapter → evaluation → Streamlit app

Visual: Figure 3 (pipeline diagram) from the project report

What to say
"The project is a pipeline. I start with structured educational content, turn it into instruction-response examples, clean and split them, and train LoRA adapters on a 4-bit frozen model. The trained adapter is evaluated and then used by the teaching app."

Slide 5 · The instruction dataset (1:30)

Put on the slide

- 840 examples, 37 topics, 14 categories
- Format: instruction → response (+ optional input)
- Categories: definitions, analogies, ELI10, viva, interview, MCQ, numerical…
- 756 training / 84 validation (stratified)

Visual: one real JSON record from data/instruction_dataset.jsonl + the category table

What to say

"Instruction tuning needs examples of the behaviour we want. Each record is an instruction and an ideal response. The content was written as structured notes for 37 topics, and a generator script turned each note into different teaching tasks — definitions, analogies, viva feedback, MCQs. Here is a real record."

Slide 6 · Key concepts (1:00)

Put on the slide

- Fine-tuning: adapt a pretrained model
- Instruction tuning: instruction → response pairs
- SFT: learn from desired outputs
- PEFT: train only a few parameters

What to say

"Fine-tuning adapts a pretrained model to a task. Instruction tuning is fine-tuning on instruction–response pairs. It is supervised because every example contains the desired answer. And PEFT means we don't update all the weights — only a small number of extra parameters."

Slide 7 · LoRA and QLoRA (1:30)

Put on the slide

- LoRA: $h = Wx + (\alpha/r)\cdot BAx$ — W frozen, A and B trained
- $r = 16$, $\alpha = 32$ → 18.5 M trainable of 1.56 B (1.18%)
- QLoRA: base model stored in 4-bit NF4 (~1 GB instead of ~3 GB)
- Adapters trained in bf16; paged 8-bit optimiser

Visual: Figure 1 (LoRA diagram) and Figure 2 (memory chart)

What to say

"LoRA keeps the original weight matrix frozen and learns two small matrices, A and B, whose product is the update. With rank 16 that is only 1.18 percent of the parameters. QLoRA additionally stores the frozen model in 4-bit, which is about four times smaller, so everything fits on my 8 GB laptop GPU."

Slide 8 · Training (1:00)

Put on the slide

- Qwen2.5-1.5B-Instruct, 3 epochs, LR 2e-4, effective batch 8
- 16.5 minutes, peak 2.81 GB GPU memory
- Loss computed on responses only
- Output: 46 MB adapter (model is ~3 GB)

Visual: results/training_loss.png

What to say

"I trained with Hugging Face TRL's SFTTrainer. Training took about seventeen minutes and never used more than three gigabytes of GPU memory. The loss curve shows validation loss dropping and then levelling off around step 175 — so two epochs would have been enough."

Slide 9 · Results (1:30)

Put on the slide

- Validation perplexity: 8.612 → 2.482 (−71%)
- MCQ format: 2/2 parseable vs 0/2 for the base model
- Consistent teaching structure, also on unseen topics
- Not always better — shorter, can still make mistakes

Visual: Figure 6 (per-category chart) + one side-by-side example

What to say

"On held-out examples, perplexity fell by seventy-one percent — the model learned the target teaching style. It also reliably follows formats: asked for MCQs, it gives answer keys; the base model doesn't. I don't claim it is better at everything: its answers are shorter, and it can still make factual mistakes."

Slide 10 · Live demo (4:00)

Put on the slide

- (switch to the app — see Section 4)

What to say

"Now let me show you the system working live."

Slide 11 · Limitations and future work (0:45)

Put on the slide

- Split by example, not topic — validation overlaps training topics
- Instruction tuning changes style, not knowledge
- No human evaluation yet
- Future: topic-level split, DPO, RAG over textbooks, bigger model

What to say

"The main limitations: validation examples share topics with training, so the metrics show style learning rather than new knowledge, and the model can still be wrong. Next steps are a stricter split, preference tuning with DPO, and retrieval from textbooks to improve accuracy."

Slide 12 · Conclusion (0:30)

Put on the slide

- The one-sentence summary
- Thank you — questions?

What to say

"To conclude: I took a pretrained language model and specialised it into an AI teaching assistant using instruction tuning with LoRA and QLoRA, trained on a laptop in under twenty minutes. Thank you — I'm happy to take questions."

4. The Live Demo (4 minutes)

The demo is where you score the most marks, and also where things can go wrong. Practise it until you can do it without thinking. Start the app **at least 2 minutes before** you present, because loading the model takes about 15 seconds on first use.

```
venv\Scripts\activate
streamlit run app.py        # opens http://localhost:8501
```

Demo step 1 · Show the dataset (20 s)	
Do	Switch to VS Code and show `data/instruction_dataset.jsonl`. Point at one line.
You should see	One JSON record per line with instruction, input, output and category.
Say	*“This is the instruction dataset the model was trained on — 840 examples like this one.”*
If it goes wrong	Show the example record on your dataset slide instead.

Demo step 2 · Base vs fine-tuned (90 s) — the most important step	
Do	App → **Base vs Fine-Tuned**. Type *Explain RNN to a beginner.*, keep “Deterministic decoding” ticked, click **Compare**.
You should see	Two columns: BASE MODEL (longer, conversational) and FINE-TUNED MODEL (structured, with a “Key idea to remember” line). Below them: a list of observable differences.
Say	*“Both answers come from the same model and the same prompt. The only difference is my 46 MB adapter, which is switched off for the left column. You can see the learned teaching structure on the right.”*
If it goes wrong	Scroll down to “Saved evaluation results” and pick a saved question — these were generated earlier and always work.

Demo step 3 · Explain Like I'm 10 (40 s)	
Do	App → **AI Tutor**. Sidebar teaching mode: **Explain Like I'm 10**. Ask *What is an LSTM?*
You should see	A short analogy-based answer ending with “The grown-up version: …”.
Say	*“The teaching mode changes the instruction sent to the model — and the model was trained on exactly this instruction format.”*
If it goes wrong	Use the quick action button “□ Give Analogy” instead.

Demo step 4 · Viva mode (60 s)	
Do	App → **Viva Mode**. Choose a topic (e.g. LSTM) in the sidebar. Click **Ask me a question**, type a deliberately partial answer, and click **Submit answer**.
You should see	The examiner's question, then feedback with a verdict, what you got right and a model answer.
Say	*“The model learned to act as an examiner from feedback examples in the dataset, where the student's answer is given in the input field.”*
If it goes wrong	If the question is odd, click the button again for a new one.

Demo step 5 · Quiz / MCQ (40 s)	
Do	App → **Quiz Mode**, click **New Question**, pick an answer, click **Submit**. Optionally: **MCQ Generator** → RNN, 5 questions, Medium.
You should see	A question with options A–D, then “Correct!” or the right answer with an explanation, and the score.
Say	*“The fine-tuned model writes MCQs in a fixed format with an answer key, so the app can grade them. The base model doesn't reliably do that.”*
If it goes wrong	If a generated question looks wrong, switch the question source to **Curated question bank** and say: “model-generated questions can contain errors — that's a known limitation.”

Demo step 6 · Training evidence (20 s)	
Do	Show `config.py` (LoRA settings) and `results/training_loss.png`.
You should see	LORA_R = 16, LORA_ALPHA = 32, USE_4BIT = True; the loss curve.
Say	*“These are the actual settings: rank 16 on all attention and MLP layers, 1.18 percent trainable.”*
If it goes wrong	Use the figures on your slides.

Golden rules for the demo

- Use **Base vs Fine-Tuned with deterministic decoding** for the comparison — the result is repeatable.
- **Never re-run training** during the presentation; it takes 17 minutes and overwrites the adapter.
- Close other GPU-heavy programs (games, video editors) before starting.
- If the app fails completely, don't debug in front of the faculty. Say “I'll show the saved results”, and use the project report figures and `results/comparison.json`.
- Take your own screenshots of every demo step the day before, as a last-resort backup.

5. Talking About the Dataset and AI Assistance Honestly

Faculty often ask **where the training data came from** and **what you did yourself**. Answer accurately — it protects you and it is expected in academic work.

5.1 The facts

- The educational content (definitions, analogies, viva and interview Q&A, MCQs, numerical problems) is written as structured notes in `scripts/knowledge/*.py`.
- Those notes were **written with the help of an AI coding assistant (Claude)**, based on standard textbook material and the original papers (e.g. LSTM 1997, Transformer 2017, LoRA 2021, QLoRA 2023).
- A generator script converts the notes into 840 training examples. It does not look anything up; it only rearranges the written notes into different task formats.
- The model was then genuinely fine-tuned on that data on your laptop, and the results are real measurements.

5.2 How to say it

Suggested wording

“The training content was written as a structured knowledge base for 37 topics, with the help of an AI assistant, based on standard textbook material, and I reviewed it. A generator script then converted it into 840 instruction–response examples in 14 teaching styles. The fine-tuning, evaluation and application were run on my own laptop, and all the numbers I show come from those runs.”

Important — do this before the presentation

Only say “I reviewed it” if you actually have. Before the day, read through the topics you are most likely to be asked about (RNN, LSTM, CNN, overfitting, LoRA, QLoRA, instruction tuning) in `scripts/knowledge/` and compare them with your textbook or lecture notes. If your institution has rules about declaring AI assistance, follow them — mention it in your report and slides as well.

5.3 Things NOT to claim

Don't say	Why	Say instead
“The data was collected from the internet.”	It wasn't — it was written as structured notes.	“It was written as a structured knowledge base and converted by a generator.”
“The model is always correct.”	It isn't; instruction tuning changes style, not knowledge.	“It follows the teaching style reliably, but it can still make mistakes.”
“It is better than ChatGPT.”	Never measured, and not true for general knowledge.	“It is specialised for a teaching style that runs locally on a laptop.”
“Accuracy is 97%.”	Accuracy is not a valid metric for free-text answers.	“Perplexity dropped from 8.612 to 2.482.”
“I trained a model from scratch.”	The base model was pre-trained by the Qwen team.	“I fine-tuned a pretrained model, training 1.18% of its parameters.”

6. Questions Faculty Are Likely to Ask

Short, confident answers are best. If you don't know something, say so and explain how you would find out.

Q. Is this just ChatGPT with a prompt?

No. No external API is used. The model runs locally, and I changed its behaviour by training: 18,464,768 LoRA parameters were learned from my dataset. You can see the adapter file and the training logs.

Q. What exactly did you train?

Only the LoRA adapter matrices — 1.18% of the parameters — on the attention and MLP layers of all 28 layers. The 1.5-billion-parameter base model stayed frozen in 4-bit.

Q. Where did the data come from?

It was written as structured notes for 37 topics with AI assistance, based on textbook material, and a generator converted it into 840 examples in 14 categories (see Section 5).

Q. Why only 840 examples? Is that enough?

Instruction tuning is about teaching behaviour and format, and research such as the LIMA paper shows that a few hundred to a few thousand high-quality examples are enough for that. The loss curve plateaued, which suggests more epochs wouldn't help; more diverse data would.

Q. How do you know the model improved?

Validation perplexity on held-out answers fell from 8.612 to 2.482, and in side-by-side tests the fine-tuned model follows the trained formats — for example 2 out of 2 MCQs with answer keys versus 0 for the base model.

Q. Why didn't you use accuracy?

Many different explanations are correct, so exact-match accuracy is meaningless for free text. Perplexity measures how well the model predicts reference answers, and I compared answers qualitatively as well.

Q. Did the model overfit?

Validation loss plateaued from about step 175 while training loss kept falling — the start of memorisation — but validation loss never went up. Early stopping at 2 epochs would give the same result.

Q. What is the difference between LoRA and QLoRA?

Both train small adapters on a frozen model. QLoRA stores the frozen model in 4-bit NF4 instead of 16-bit, cutting its memory about four times.

Q. Why does 4-bit not destroy accuracy?

The weights are only stored in 4-bit; they are converted back to 16-bit for every calculation. They are frozen, so errors don't accumulate, and the adapters can compensate.

Q. Why Qwen2.5-1.5B?

It is open-source, instruction-capable, strong for its size, and fits QLoRA training on an 8 GB laptop GPU. The model name is one setting in config.py, so it can be swapped.

Q. What was your contribution?

Designing the pipeline, preparing and reviewing the dataset, configuring and running the QLoRA training, evaluating base versus fine-tuned, and building the teaching application. (Adjust this to what you personally did.)

Q. What is novel here?

The techniques are established; the contribution is applying them end-to-end to a specific educational use case, on consumer hardware, with an honest evaluation and a usable application.

Q. What happens when the model is wrong?

The app shows a disclaimer to verify with textbooks, the quiz has a curated question bank, and future work includes retrieval from trusted textbooks (RAG) to ground answers.

Q. Could this work with a bigger model?

Yes — change MODEL_NAME in config.py (e.g. Qwen2.5-3B) and retrain; QLoRA makes 3B feasible on 8 GB.

Q. How long would full fine-tuning take instead?

Full fine-tuning needs about 16 bytes per parameter — roughly 25 GB for this model — so it would not even fit on this GPU. That is exactly why PEFT and QLoRA are used.

7. Presentation Do's and Don'ts

Do	Don't
Start with the problem and your one-sentence summary	Start by listing libraries and versions
Show real outputs and real numbers	Describe results you can't show
Explain LoRA with the diagram and one equation	Put long code on slides
Mention limitations yourself	Wait for faculty to find them
Say “the model can make mistakes”	Claim the model is always correct
Keep the demo to the rehearsed steps	Improvise new questions live
Pause after key numbers (1.18%, −71%)	Rush through the results slide
Say “I don't know, but I would check…” when unsure	Guess confidently
Be clear about what AI tools helped with	Present AI-assisted work as entirely your own

8. Checklist

8.1 The day before

- Rehearse the full presentation with a timer, at least twice.
- Run `python scripts/test_model.py` — all 8 tests should PASS.
- Run the demo steps once and take screenshots of each as a backup.
- Review the knowledge-base content for the main topics (Section 5.2).
- Charge the laptop fully; the GPU is slower on battery power.

8.2 Thirty minutes before

- Plug in the charger and close other programs, especially anything using the GPU.
- Start the app: `streamlit run app.py`, and wait until a first question has been answered.
- Open the tabs listed in Section 2 (dataset, config.py, loss curve, report PDF).
- Increase the browser zoom (Ctrl +) so the faculty can read the answers.
- If the room has no internet, the app still works — the model is already downloaded. If it tries to connect, run `$env:HF_HUB_OFFLINE=1` (PowerShell) or `set HF_HUB_OFFLINE=1` (Command Prompt) before starting it.

8.3 Numbers to remember

What	Number
Base model	Qwen2.5-1.5B-Instruct (1.54 B parameters, 28 layers)
Trainable parameters	18,464,768 (1.18%)
LoRA	r = 16, α = 32, dropout 0.05, 7 target modules
Quantisation	4-bit NF4 + double quantisation, bf16 compute
Dataset	840 examples · 37 topics · 14 categories · 756 / 84 split
Training	3 epochs · 285 steps · LR 2e-4 · 16.5 min · 2.81 GB peak
Result	Perplexity 8.612 → 2.482 (−71%)
Adapter	46 MB (full model ~3 GB)

Last line before questions

“To summarise: a pretrained model, specialised into a teaching assistant with instruction tuning, training only 1.18 percent of its parameters, on a laptop, in under twenty minutes. Thank you.”